Tuwaiq CLUB

أكاديمية طويق
Tuwaiq Academy

KFUPM
Student Affairs



DAY 2

Core API

CRUD with function views, path() URLs, and manual conversion

Monday, October 26, 2026

Your five-day journey

Yesterday the Task model persisted. Today it becomes an API.

01

Foundation

Project, Task model, migrations, admin

DONE

02

Core API

CRUD with function views

TODAY

03

Serialization & validation

Serializers, rules, clear field errors

04

Security & configuration

Tokens, ownership, environment

05

Publish

Deploy, verify, README, live demo

Today's session

Learn the API pattern, practice it, then apply it to the project

Day 1 push
first 10 min

Explain + live demo
~60 min

Exercise
~60 min

Break
short

Project
~60 min

Exit check
end of day

Priority rule: finish the exercise core, then move to the project. Stretch work can wait.

Learning outcomes

By the end of today you can...

01 Convert model objects to JSON-ready dictionaries

02 Write one function view per CRUD operation

03 Connect views with path(), URL names and include()

04 Read status codes: 200, 201, 204, 400, 404 and 405

The core API request path

Each layer has one job in a CRUD request

URL

path() matches the request

Function view

Reads request.data and runs the ORM

Response

dict or list becomes JSON + status

URL -> view -> ORM -> database -> dict -> Response -> JSON

A function view

days/day-02/example/views.py

```
@api_view(["GET"])
def get_movie(request, pk):
    movie = get_object_or_404(Movie, pk=pk)
    return Response(movie_to_dict(movie))
```

API tool	Job
`@api_view([...])`	Allowed methods; other methods return 405
`request.data`	The parsed JSON request body
`Response(data, status=...)`	JSON output and HTTP status (200 by default)

Object to dict by hand

Choose the JSON keys explicitly before returning the response

```
def movie_to_dict(movie):
    return {
        "id": movie.id,
        "title": movie.title,
        "genre": movie.genre,
        "release_date": movie.release_date,
        "director": movie.director_id,
        "director_name": movie.director.name,
        "created_at": movie.created_at,
    }
```

Project: task_to_dict returns 8 fields, with owner as its user id.

Routing with path()

days/day-02/example/urls.py; the project mounts these URLs under api/

```
urlpatterns = [
    path("movies/", views.list_movies, name="movie-list"),
    path("movies/create/", views.create_movie, name="movie-create"),
    path("movies/<int:pk>/", views.get_movie, name="movie-detail"),
    path("movies/<int:pk>/update/", views.update_movie, name="movie-update"),
    path("movies/<int:pk>/delete/", views.delete_movie, name="movie-delete"),
]
# config/urls.py
path("api/", include("movies.urls"))
```

<int:pk> passes the number to the view. URL names work with reverse().

The CRUD contract

Five URLs, six HTTP operations on the Task resource

Method	URL	View	Success
GET	`/api/tasks/`	`list_tasks`	200
POST	`/api/tasks/create/`	`create_task`	201
GET	`/api/tasks/{id}/`	`get_task`	200
PUT	`/api/tasks/{id}/update/`	`update_task`	200
PATCH	`/api/tasks/{id}/update/`	`update_task`	200
DELETE	`/api/tasks/{id}/delete/`	`delete_task`	204

One POST request, step by step

POST /api/movies/create/ with title, imdb_code and director

01 path() selects create_movie; @api_view allows POST

02 request.data contains the parsed JSON body

03 Manual check: an empty title returns 400; nothing is saved

04 The ORM creates the row; added_by = request.user

05 Response(movie_to_dict(movie), status=201) returns JSON

Only the rules we explicitly code are checked today.

PUT and PATCH

The intended meaning is different; today the implementation is deliberately simple

PATCH: partial update

Send only the fields that change.
{"status": "DONE"}

PUT: full replacement

Send the complete writable object.
Every required field belongs in the body.

Day 2: both change only the fields sent. Day 3 makes PUT strict with partial=False.

Status codes

Read the status and the response body together

Code	Meaning	Example
200	OK	List, retrieve or update
201	Created	A new row was saved
204	No Content	Deleted; the response body is empty
400	Bad Request	A manual input check failed
404	Not Found	get_object_or_404 cannot find the row
405	Method Not Allowed	DELETE sent to the retrieve URL

What slips through

The remaining input problems are the reason for Day 3

```
POST /api/tasks/create/
{"title": "X", "status": "FINISHED"}
```

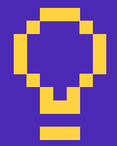
Bad status

FINISHED is saved even though it is not a valid choice.

Bad date

"due_date": "tomorrow" can reach the database and fail with 500.

Tomorrow: one serializer handles conversion and validates every input field.



LIVE DEMO

The browsable API

10 min: every endpoint, then the gaps in manual checks

CRUD	404	405	Bad input
list to delete	unknown id	wrong method	status slips through

Watch the status code and response body every time.

HOUR 2 · EXERCISE

Book CRUD API

Five function views and five URLs

Core	Time	Stretch
Tasks 1-4	about 50 min	Task 5

Start in days/day-02/exercise/; use the self-check requests.

Exercise Task 1: book_to_dict

catalog/views.py; return exactly these 8 keys in this order

JSON key	Value
`id, title, isbn`	`book.id, book.title, book.isbn`
`published_year`	`book.published_year`
`available_copies`	`book.available_copies`
`author`	`book.author_id`
`author_name`	`book.author.name`
`created_at`	`book.created_at`

Given: author_to_dict and list_authors show the complete pattern.

Exercise Tasks 2-3: the five views

catalog/views.py; replace the 501 placeholders

Task	Build	Check
2a	`list_books`	200 + the 6 sample books
2b	`create_book`	201; missing fields -> 400; unknown author -> 404
3a	`get_book`	200, or 404 for an unknown id
3b	`update_book`	PUT/PATCH; change only fields that were sent
3c	`delete_book`	204; a later GET returns 404

Create requires title, isbn, published_year and author. The server chooses id.

Exercise Task 4: URLs; Task 5: stretch

catalog/urls.py; every book view needs its own path()

URL under /api/	Name
`books/`	`book-list`
`books/create/`	`book-create`
`books/<int:pk>/`	`book-detail`
`books/<int:pk>/update/`	`book-update`
`books/<int:pk>/delete/`	`book-delete`

Stretch 5A: Meta.ordering = ["title"]; makemigrations + migrate.
Stretch 5B: complete crud_checklist.md (9 requests + 3 questions).

Complete the 8 core self-check requests before the project.

BREAK

Short break

After the break: apply the same pattern to the Task API

HOUR 3 · PROJECT

CRUD for the Task API

Apply the function-view pattern to your own project

Commit Day 1	Checkpoint	P1-P8	9 requests
save your work	day-02	build CRUD	verify + push

Keep Git history, .venv and database. Tokens arrive on Day 4.

Day 2 project increment

P1-P8; log in through /api-auth/login/ for now. Tokens arrive on Day 4.

TODO	What you build
P1	Newest-first ordering; makemigrations + migrate
P2	task_to_dict: 8 API fields; owner as its id
P3	List + create; blank title -> 400; owner = request.user
P4	get_task; unknown id -> 404
P5	update_task; change only the fields sent
P6	delete_task; return 204
P7	Five named path() routes in tasks/urls.py
P8	Include tasks.urls under api/; DRF login under api-auth/

Project checklist: requests 1-5

Log in as alice; check the database after each write

#	Request	Expected
1	`GET /api/tasks/`	200; newest first
2	`POST /api/tasks/create/`	201; owner = alice, 999 ignored
3	`GET /api/tasks/{id}/`	200
4	`PUT /api/tasks/{id}/update/`	200; supplied fields changed
5	`PATCH /api/tasks/{id}/update/`	200; only status changes

```
POST: {"title": "Prepare slides", "status": "TODO", "owner": 999}
PUT: {"title": "Prepare final slides", "description": "",
      "status": "IN_PROGRESS", "due_date": null}
PATCH: {"status": "DONE"}
```

Project checklist: requests 6-9

Use the id created in request 2

#	Request	Expected
6	`DELETE /api/tasks/{id}/delete/`	204; row removed
7	`GET /api/tasks/{id}/`	404; the row is gone
8	`POST /api/tasks/create/` `{"title": "   "}`	400; Title is required.
9	`DELETE /api/tasks/{id}/`	405; wrong URL for delete

Validation arrives on Day 3; isolation and tokens arrive on Day 4.

Day 2 exit check

Ready for the next day when every check passes

01 Five URLs reach the right views; a wrong method returns 405

02 Create -> 201; retrieve -> 200; unknown id -> 404

03 PUT/PATCH change intended fields; delete -> 204

04 The server assigns owner; a supplied owner is ignored

05 Blank title -> 400; nothing is saved

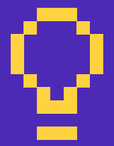
06 All 9 checklist requests pass; the commit is pushed

Next: replace manual conversion and checks with a serializer

Tuwaiq CLUB

أكاديمية طويق
Tuwaiq Academy

KFUPM
Student Affairs



Day 2 complete

The Task API now supports CRUD with function views and explicit URLs.

Next: serialization, validation, and clear field errors